

CS102L Data Structures & Algorithms

Lab 12



DEPARTMENT OF COMPUTER SCIENCE
DHANANI SCHOOL OF SCIENCE AND ENGINEERING
HABIB UNIVERSITY
SPRING 2026

Contents

1	Introduction	2
1.1	Instructions	2
1.2	Marking scheme	2
1.3	Lab Objectives	2
1.4	Late submission policy	2
1.5	Use of AI	3
1.6	Viva	3
2	Prerequisites	4
2.1	Pytest	4
3	Lab Exercises	5
3.1	Helper Functions	6
3.1.1	Testing	8
3.2	Question 1	8
3.2.1	Testing	9
3.3	Question 2	9
3.3.1	Testing	9

Introduction

1.1 Instructions

- This lab will contribute 1% towards your final grade.
- The deadline for submission of this lab is at the end of lab time.
- The lab must be submitted online via CANVAS. You are required to submit a .zip file that contains all the .py files.
- The .zip file should be named as *lab12_aa12345.zip* where *aa12345* will be replaced with your student id.
- **Files that don't follow the appropriate naming convention will not be graded.**
- **Your final grade will comprise of both your submission and your lab performance.**

1.2 Marking scheme

This lab will be marked out of 100.

- 50 Marks are for the completion of the lab.
- 50 Marks are for progress and attendance during the lab.

1.3 Lab Objectives

The objective of this lab is:

- To learn how to create Binary Search Tree in Python

1.4 Late submission policy

There is no late submission policy for the lab.

1.5 Use of AI

Taking help from any AI-based tools such as ChatGPT is strictly prohibited and will be considered plagiarism.

1.6 Viva

Course staff may call any student for Viva to provide an explanation for their submission.

Prerequisites

2.1 Pytest

Pytest is a popular testing framework in Python known for its simplicity and power in facilitating efficient and scalable testing for applications and software. In order to use Pytest, one first needs to install it and create test functions prefixed with *test_* in a Python file. To install Pytest, you can write the following command on the terminal:

```
pip install pytest
```

After installing Pytest, one can execute all the tests by typing the following command on the terminal:

```
pytest
```

If you want to test any particular file, then you will have to specify the filename as follows:

```
pytest <filename>.py
```

In case the above mentioned pytest commands do not work on your system, even though pytest is installed, try using the following commands:

```
py -m pytest
```

```
py -m pytest <filename>.py
```

If all the tests pass successfully, then you should receive a similar output.

```
PS C:\Work\Spring2024\DSA\Lab1> pytest test_Question1.py
===== test session starts =====
platform win32 -- Python 3.11.2, pytest-7.4.4, pluggy-1.3.0
rootdir: C:\Work\Spring2024\DSA\Lab1
collected 10 items

test_Question1.py .....

===== 10 passed in 0.06s =====
```

However, if all the tests fail, then you should receive a similar output.

```
PS C:\Work\Spring2024\DSA\Lab1> pytest test_Question1.py
===== test session starts =====
platform win32 -- Python 3.11.2, pytest-7.4.4, pluggy-1.3.0
rootdir: C:\Work\Spring2024\DSA\Lab1
collected 10 items

test_Question1.py FFFFFFFF

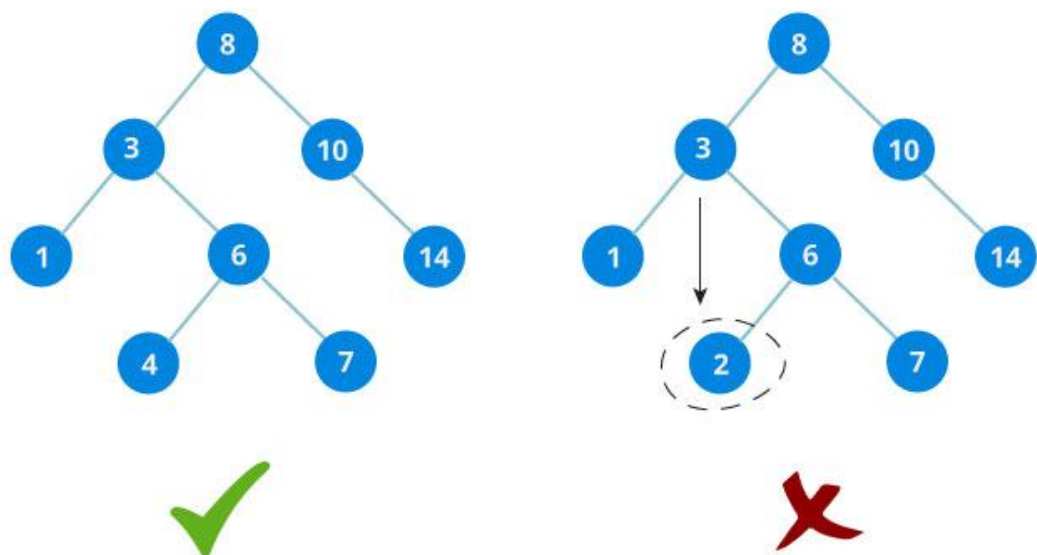
===== FAILURES =====
test_question1[1st10-1st20-e54dc15ccafa608142ffa6340b035c327f972e24882052cfc3345f240f4ee237] ____
```

Lab Exercises

Binary Search Tree, is a node-based binary tree data structure which has the following properties:

- The left subtree of a node contains only nodes with keys lesser than the node's key.
- The right subtree of a node contains only nodes with keys greater than the node's key.
- The left and right subtree each must also be a binary search tree. There must be no duplicate nodes.

The above properties of Binary Search Tree provide an ordering among keys so that the operations like search, minimum and maximum can be done fast. If there is no ordering, then we may have to compare every key to search a given key.



This Binary Search Tree shown in Fig 3.1 has resulted from inserting the following keys in the order given: 8, 3, 10, 1, 6, 14, 4, 7 and 13. We can represent the Binary Search Tree using Nested Dictionary in Python.

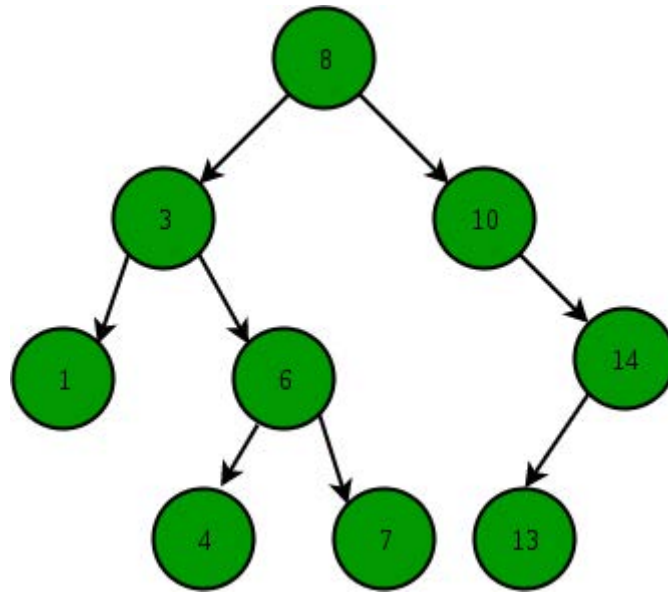


Figure 3.1: Resultant Binary Search Tree.

```

{'value': 8, 'right': {'value': 10, 'left': {}, 'right':
{'value': 14, 'left': {'value': 13, 'left': {}, 'right':
{}}, 'right': {}}, 'left': {'value': 3, 'left': {'value':
1, 'left': {}, 'right': {}}, 'right': {'value': 6, '
left': {'value': 4, 'left': {}, 'right': {}}, 'right': {'
value': 7, 'left': {}, 'right': {}}}}

```

3.1 Helper Functions

This question should be implemented in the file **helper_functions.py**.

Implement the following helper functions:

- **insert(bst, key)**: A new key is always inserted at leaf. We start searching a key from root till we hit a leaf node. Once a leaf node is found, the new node is added as a child of the leaf node. Implement the function **insert(bst, key)** that takes a Binary Search Tree and key as a parameter and inserts key into the Binary Search Tree.
- **exist(bst, key)**: To search a given key in Binary Search Tree, we first compare it with root, if the key is present at root, we return root. If key is greater than root's key, we recur for right subtree of root node. Otherwise we recur for left subtree. Implement the function **exist(bst, key)** that takes a Binary Search Tree and key as a parameter and returns **True** if the key exists; **False** otherwise.
- **minimum(bst, starting_node)**: Just traverse the node from root to left recursively until left is **NULL**. The node whose left is **NULL** is the node with

minimum value. Implement the function `minimum(bst, starting_node)` that takes a Binary Search Tree & a starting node as a parameter and returns the minimum node in the BST from the starting node.

- `maximum(bst, starting_node)`: Just traverse the node from root to right recursively until right is `NULL`. The node whose right is `NULL` is the node with maximum value. Implement the function `maximum(bst, starting_node)` that takes a Binary Search Tree and a starting node as a parameter and returns the maximum node in the BST from the starting node.
- `inorder_traversal(bst, res)`:
 1. Traverse the left subtree, i.e., call `Inorder(left-subtree)`.
 2. Visit the root.
 3. Traverse the right subtree, i.e., call `Inorder(right-subtree)`.

Implement the function `inorder_traversal(bst, res)` that takes a Binary Search Tree and a `res` array and stores in-order traversal of the Binary Search Tree in the `res` array.

- `preorder_traversal(bst, res)`:
 1. Visit the root.
 2. Traverse the left subtree, i.e., call `Preorder(left-subtree)`.
 3. Traverse the right subtree, i.e., call `Preorder(right-subtree)`.

Implement the function `preorder_traversal(bst, res)` that takes a Binary Search Tree and a `res` array and stores the pre-order traversal of the Binary Search Tree.

- `postorder_traversal(bst, res)`:
 1. Traverse the left subtree, i.e., call `Postorder(left-subtree)`.
 2. Traverse the right subtree, i.e., call `Postorder(right-subtree)`.
 3. Visit the root.

Implement the function `postorder_traversal(bst, res)` that takes a Binary Search Tree and a `res` array and stores the post-order traversal of the Binary Search Tree.

- `successor(bst, key)`: In a Binary Search Tree, the In-order Successor of a given node is defined as the node with the smallest value greater than the value of the given node ([Learn more here](#)).

Implement the function `successor(BST, key)` that takes a Binary Search Tree and a `key` and returns the successor of that key.

Pseudocode 1 Find In-order Successor in BST

Require: *BST*: Binary Search Tree, *key*: Key to find the successor of

Ensure: The in-order successor node if it exists, otherwise None

- 1: Locate the node with the given *key* in the tree.
 - 2: **if** the node has a right child **then**
 - 3: Move to the right child.
 - 4: From there, keep going left until you reach the leftmost node.
 - 5: This leftmost node is the in-order successor — return it.
 - 6: **else**
 - 7: Start again from the root node.
 - 8: As you search for the key, keep track of the last node where you turned left.
 - 9: That last left turn node is the in-order successor.
 - 10: **end if**
 - 11: If no such node is found, return None.
-

3.1.1 Testing

To test your function, type the following command in the terminal:

```
py -m pytest tests/test_helper_functions.py
```

3.2 Question 1

This question should be implemented in the file **q1.py**.

Implement the following helper functions:

1. **q_1a()**: Using the helper function **insert(bst, key)**, create the binary search tree that results from inserting the following keys in the order given: 68, 88, 61, 89, 94, 50, 4, 76, 66, and 82.
2. **q_1b(bst)**: Using the helper function **exist(bst, key)**, check whether key 50 exists in resultant Binary Search Tree.
3. **q_1c(bst)**: Using the helper function **exist(bst, key)**, check whether key 49 exists in resultant Binary Search Tree.
4. **q_1d(bst)**: Using the helper function **minimum(bst, starting_node)**, find the node with the minimum value in resultant Binary Search Tree from starting node = 68.
5. **q_1e(bst)**: Using the helper function **minimum(bst, starting_node)**, find the node with the minimum value in resultant Binary Search Tree from starting node = 88.

6. `q_1f(bst)`: Using the helper function `maximum(bst, starting_node)`, find the node with the maximum value in resultant Binary Search Tree from starting node = 68.
7. `q_1g(bst)`: Using the helper function `maximum(bst, starting_node)`, find the node with the maximum value in resultant Binary Search Tree from starting node = 61.
8. `q_1h(bst)`: Using the helper function `inorder_traversal(bst, result)`, perform in-order traversal of the Binary Search Tree.
9. `q_1i(bst)`: Using the helper function `preorder_traversal(bst, result)`, perform pre-order traversal of the Binary Search Tree.
10. `q_1j(bst)`: Using the helper function `postorder_traversal(bst, result)`, perform post-order traversal of the Binary Search Tree.
11. `q_1k(bst)`: Using the helper function `successor(BST, key)`, find the successor of 76.

3.2.1 Testing

To test your function, type the following command in the terminal:

```
py -m pytest tests/test_q1.py
```

3.3 Question 2

This question should be implemented in the file `q2.py`.

In this question, you should create a new Binary Search Tree and organize input strings in the list `["begin", "do", "else", "end", "if", "then", "while"]` such that the keyword in the root is alphabetically bigger than all the keywords in the left subtree and smaller than all the keywords in the right subtree (and this holds for all nodes). Lastly, return this updated BST.

3.3.1 Testing

To test your function, type the following command in the terminal:

```
py -m pytest tests/test_q2.py
```